

Experiment - 8

AIM- To generate PWM signals using a timer and controlling the angular position of a servo motor.

APPARATUS- STM32 Discovery Kit, STM32 CUBE IDE, STM32 CubeMX, Realterm, Servo Motor, connecting wires.

PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L475VGT6.
3. **Clock configuration:** Set the clock configuration to HSI ☐ PLLCLK ☐ HCLK = 80MHz.

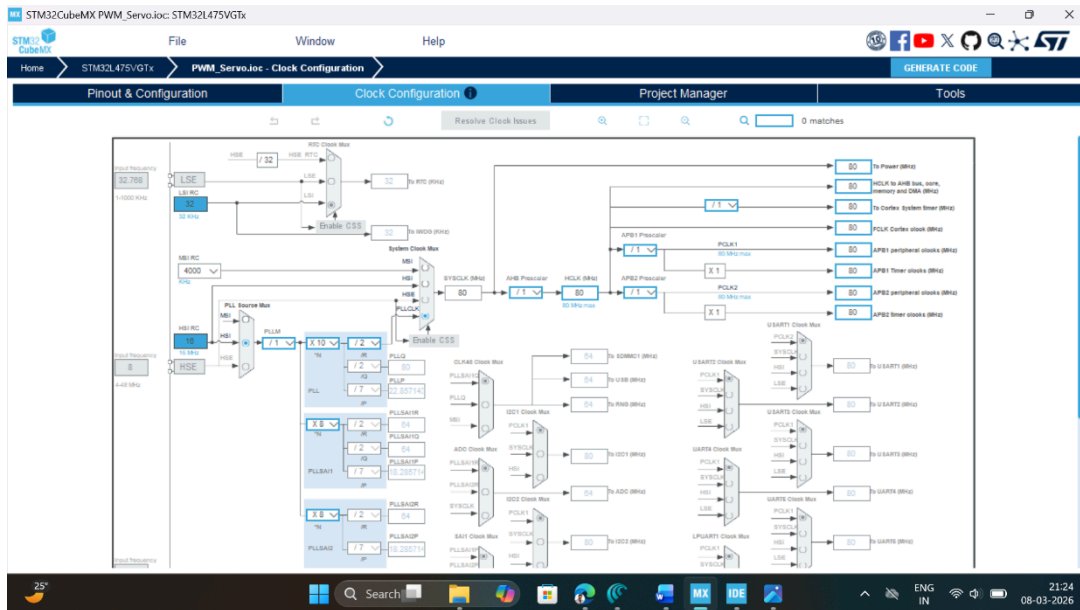


Fig-8.1 – Clock Configuration

4. **TIMER SELECTION:** Select TIMER from Timer tab as TIM2_CH1. Enable Channel 1 for PWM generation.

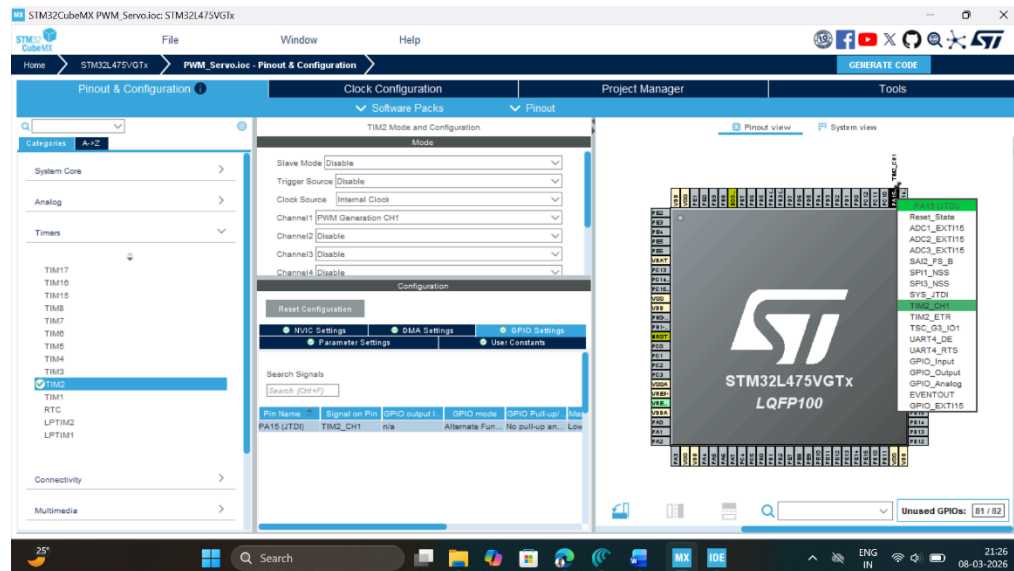


Fig 8.2 – Timer Selection

5. Under parameter setting for Timer 2 make Prescaler as 79 and Auto Reload Register as 19999, Enable Auto-reload preload.

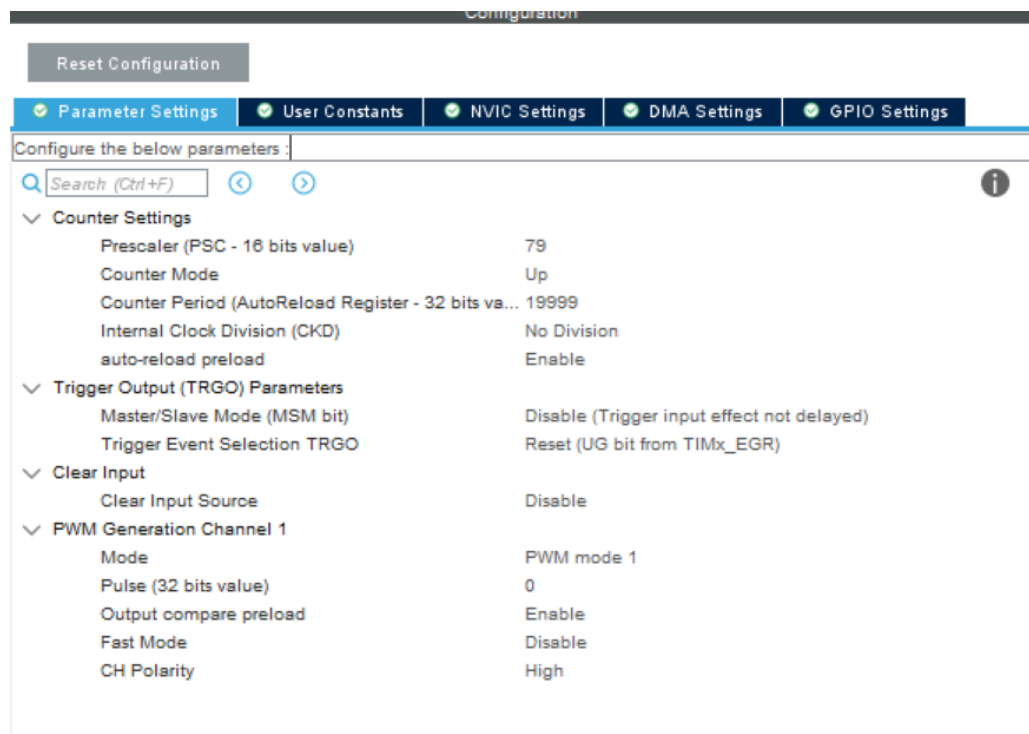


Fig-8.3 – TIM2 Parameters settings

6. Write Project name and select CubeIDE.
7. Click on generate code.
8. Click on open project.

9. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
10. Click on SRC and make the changes in **main.c** file as given below:

```
/* USER CODE BEGIN PD */
#define SERVO_MIN_PULSE 1000 // 1ms at 1MHz tick
#define SERVO_MAX_PULSE 2000 // 2ms
/* USER CODE END PD */

/* USER CODE BEGIN PFP */
void set_servo_angle(uint8_t angle); // angle 0-180
/* USER CODE END PFP */

/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
/* USER CODE END 2 */

While (1)
/* Sweep demo: 0° to 180° */
    for(uint8_t angle = 0; angle <= 180; angle += 30) {
        set_servo_angle(angle);
        HAL_Delay(1000);
    }
    for(uint8_t angle = 180; angle > 0; angle -= 30) {
        set_servo_angle(angle);
        HAL_Delay(1000);
    }

/* USER CODE BEGIN 4 */
void set_servo_angle(uint8_t angle) {
    uint32_t pulse;
    if (angle < 0) angle = 0;
    if (angle > 180) angle = 180;
    pulse = SERVO_MIN_PULSE + (SERVO_MAX_PULSE -
SERVO_MIN_PULSE) * angle / 180;
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
}
/* USER CODE END 4 */
```

11. Hardware setup: Connect STM32L475 IoT node with servo motor as given below:
 - a. Servo brown/black wire to GND.
 - b. Servo red wire to 5V (board provides 5V output).
 - c. Servo orange/yellow wire to D9 (PA15).

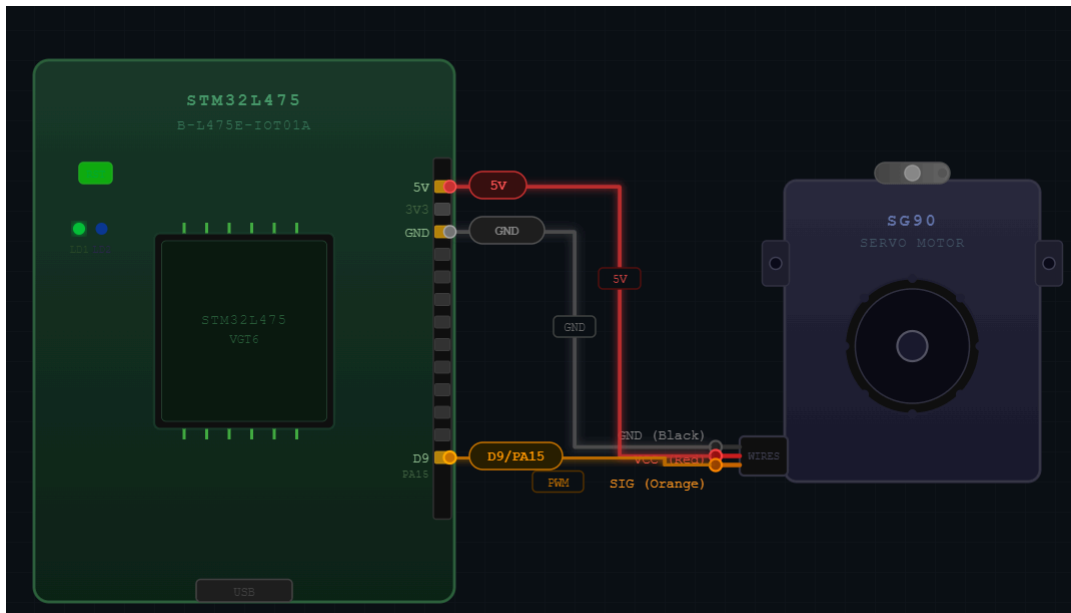
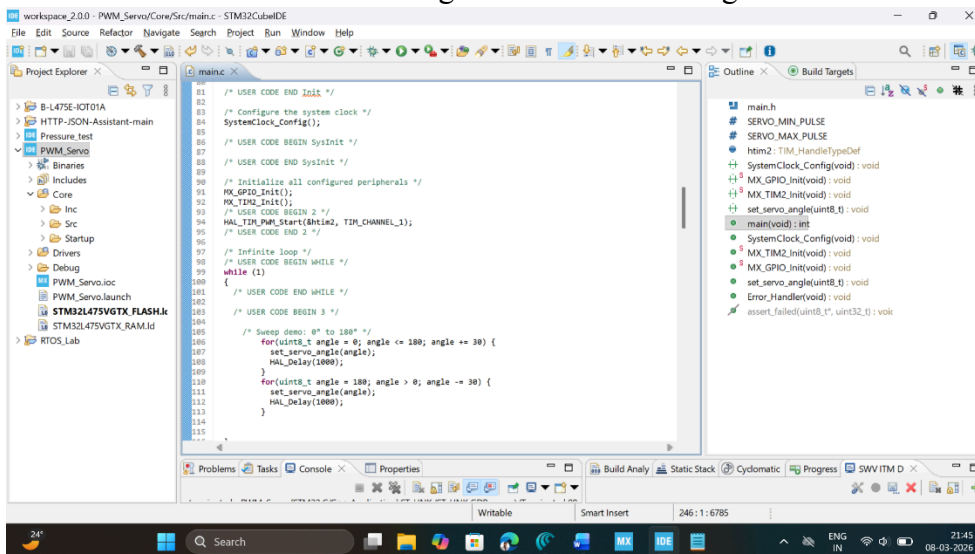


Fig 8.4: Hardware Setup

12. Now connect the board and click on “Build”, “debug”.

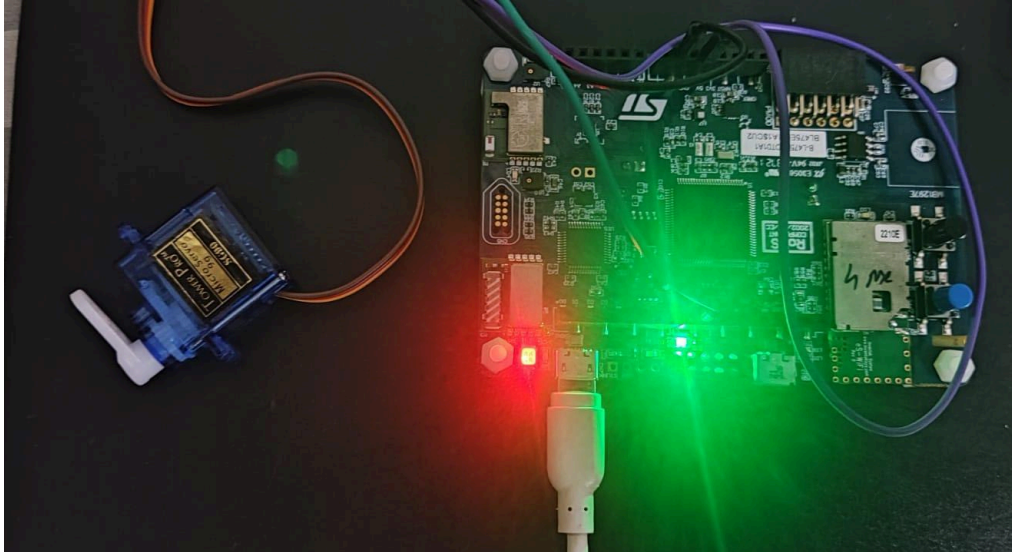


Fig 8.5: Build and Debug Tab





OBSERVATION: The angular position of a SG90 servo motor is accurately controlled by varying the Pulse-width modulation signal generated by STM32 development board using TIM2.



Reflection Summary

1. Reflect on the role of the STM32 timer's prescaler ($PSC=79$) and auto-reload register ($ARR=19999$) in generating the precise 20 ms period required for servo compatibility.

The **prescaler (PSC)** is used to reduce the high-frequency clock of the STM32 timer to a lower, manageable frequency. This allows precise timing control required for PWM generation.

The **auto-reload register (ARR)** defines the total counting range of the timer, which determines the overall period of the PWM signal.

Together, PSC and ARR control the **frequency and period of the PWM signal**, ensuring that a stable and accurate signal is generated.

For servo motors, maintaining the correct PWM period is essential, as they require a fixed timing signal to operate properly.

Conclusion:

PSC controls the timer speed, while ARR controls the PWM period, and both are essential for generating a stable signal suitable for servo motor control.

2. How did varying the PWM pulse width from 1 ms to 2 ms affect the servo motor's angular position, and what does this demonstrate about the relationship between duty cycle and servo control?

Servo motors interpret **pulse width as position information**.

Pulse Width	Servo Angle
1 ms	0°
1.5 ms	90°
2 ms	180°

In the code:

```
pulse = SERVO_MIN_PULSE +  
        (SERVO_MAX_PULSE - SERVO_MIN_PULSE) * angle / 180;
```

This changes the **duty cycle within the 20 ms period**.

Example:

- 1 ms / 20 ms = **5% duty cycle**
- 2 ms / 20 ms = **10% duty cycle**

Observation:

Increasing the pulse width increases the servo angle.

Conclusion:

Servo position is directly controlled by **PWM duty cycle (pulse width)**.

3. How would you modify the code to control multiple servo motors simultaneously using different timer channels? What limitations might you encounter on the STM32L475?

STM32 timers support **multiple PWM channels**.

Example for TIM2:

- TIM2_CH1
- TIM2_CH2
- TIM2_CH3
- TIM2_CH4

Steps to modify code

1. Enable additional channels in **CubeMX**.

2. Start PWM on each channel:

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);  
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);  
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
```

3. Create a function for each servo:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse1);  
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, pulse2);
```

Limitations

- Limited number of **timer channels**
- **GPIO pin availability**
- Increased **CPU load**
- Power supply limitations when driving many servos
- Timer resolution constraints

4. What happens to servo behavior when the PWM frequency is accidentally set to 1 kHz instead of 50 Hz? Predict and verify the observed effects.

If PWM frequency becomes **1 kHz (1 ms period)** instead of **50 Hz (20 ms period)**:

Effects

1. Servo will **not recognize the signal properly**.
2. It may:
 - Jitter or vibrate
 - Stay at a fixed position
 - Move unpredictably
3. The pulse width becomes too short for the servo control circuit to decode.

Reason

Servos are designed to detect **control pulses every ~20 ms**.

At **1 kHz**, pulses arrive **20× faster**, confusing the internal controller.

Conclusion:

Incorrect PWM frequency leads to **unstable or no servo movement**.